

Linear Regression

Instructor: Arbish Akram

1 The Problem: What Are We Trying to Do?

In a regression problem we are given a training data set of N examples,

$$\{(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)\},$$

where:

- x_n is the **input** (also called a feature, or an independent variable) for the n -th training example. It could be a single number (e.g. the size of a house) or a vector of numbers (e.g. size, number of rooms, age of the house).
- t_n is the **target** (also called the label, or the dependent variable) that we want to predict for input x_n . This is a single real number in the regression setting described here.
- N is the **number of training examples** we have. It is a positive integer, e.g. $N = 500$.
- The subscript n ranges over $n = 1, 2, \dots, N$ and is just an index that tells us which training example we are talking about.

Our goal is to find a function that takes an input x and predicts a target value t as accurately as possible, using only the patterns present in the N training examples we were given. In *linear* regression, we assume this function has a particular simple form, which we build up next.

1.1 Basis Functions $\phi(x)$ as Features

The most intuitive way to think about a **basis function** is that it is simply a recipe for turning the raw input x into a **feature**: a single number that we believe is useful for predicting the target t . A basis function $\phi_j(x)$ is just a function that takes x in and produces one such feature value out. Since it is a function of x , we are free to choose it to compute anything we like from x : a raw coordinate of x unchanged, a power of x , a transcendental function of x , or something more elaborate.

Rather than working directly with the single raw input x , linear regression works with a whole collection of M such chosen functions of x , and stacks the feature values they produce into one vector:

$$\phi(x) = \begin{pmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{M-1}(x) \end{pmatrix}, \quad \phi(x) \in \mathbb{R}^M.$$

- $\phi_0(x), \phi_1(x), \dots, \phi_{M-1}(x)$ are the individual basis functions, i.e. the individual **feature-generating functions**. Each one is a function taking the input x and returning a single real number, that number being the value of that particular feature for that particular x . Because each ϕ_j is a function we choose, its output can be anything we want it to compute from x : for example $\phi_0(x) = 1$ (a constant function, whose output is always the same number 1 regardless of x ; this is called the **bias** or **intercept** feature), $\phi_1(x) = x$ (the identity function, which simply passes the raw input through unchanged as a feature), $\phi_2(x) = x^2$ (a function that squares its input, giving a quadratic feature, useful for a polynomial model), or more exotic functions such as a Gaussian bump centred at some point, or $\sin(x)$, or $\log(x)$.
- M is the number of basis functions, i.e. the number of features, that we chose to compute from each input. This is a design choice made by the person building the model (sometimes called “feature engineering”), not something learned automatically from the data. The functions are indexed starting from 0 (i.e. $\phi_0, \phi_1, \dots, \phi_{M-1}$), so that there are exactly M of them; this indexing-from-zero convention is common in machine learning texts.

- $\phi(x)$, written in bold or with an arrow in some textbooks, is the **feature vector** for the input x : the column vector obtained by evaluating every one of the M feature-generating functions at x and stacking their outputs on top of each other. It is an $M \times 1$ matrix (a matrix with M rows and 1 column). In other words, ϕ is the function (built out of the M smaller functions $\phi_0, \dots, \phi_{M-1}$) that converts a raw input x into its vector of M features.
- When we plug in the n -th training input, we write $\phi(x_n)$, which is again an $M \times 1$ column vector: it is the feature vector obtained by running the n -th training input through every one of the M chosen feature functions.

1.2 The Weight Vector w

The prediction our model makes for an input x is defined as a weighted sum of the basis function outputs:

$$y(x, w) = \sum_{j=0}^{M-1} w_j \phi_j(x) = w^T \phi(x).$$

- $w_0, w_1, \dots, w_{M-1}$ are the **weights** (also called parameters or coefficients) of the model. These are the numbers we do not know yet and want to learn from the training data.
- w is the column vector obtained by stacking the weights: $w = (w_0, w_1, \dots, w_{M-1})^T$, also of size $M \times 1$.
- The superscript T denotes the **transpose** of a vector or matrix; see the Recall box below if this is unfamiliar.
- $w^T \phi(x)$ is therefore a **dot product** (also called an inner product) between the weight vector and the basis-function vector, and it produces a single real number: the model's predicted target value for input x .

Recall: Transpose of a Vector

If $v = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_M \end{pmatrix}$ is an $M \times 1$ column vector, then its transpose $v^T = (v_1, v_2, \dots, v_M)$ is the corresponding $1 \times M$ row vector: the transpose operation simply lays the entries out horizontally instead of vertically (it does not change the numbers, only their arrangement). Transposing a row vector similarly turns it back into a column vector. More generally, for a matrix, transposing swaps rows and columns: the entry in row i , column j of A^T equals the entry in row j , column i of A .

Recall: Dot Product of Two Vectors

If $u = (u_1, \dots, u_M)^T$ and $v = (v_1, \dots, v_M)^T$ are two column vectors of the same length M , their dot product is the single number

$$u^T v = \sum_{i=1}^M u_i v_i = u_1 v_1 + u_2 v_2 + \dots + u_M v_M.$$

Writing it as $u^T v$ (a $1 \times M$ row vector multiplied by an $M \times 1$ column vector) is exactly matrix multiplication and it is why the transpose appears: a $1 \times M$ matrix times an $M \times 1$ matrix gives a 1×1 matrix, i.e. a scalar (an ordinary number).

So, in the expression $w^T \phi(x_n)$: we take the weight vector w , transpose it into a row vector, and multiply it (dot product) with the column vector $\phi(x_n)$, giving the scalar prediction of the model for the n -th training input.

2 The Error Function

To measure how good or bad a particular choice of weights w is, the slides define an **error function** (also called a **loss function** or **cost function**):

$$E(w) = \frac{1}{2} \sum_{n=1}^N \{t_n - w^T \phi(x_n)\}^2.$$

Let us unpack every single piece of this formula.

- $t_n - w^T \phi(x_n)$: here $w^T \phi(x_n)$ is the model's prediction for the n -th training input (as derived above), and t_n is the true, known target value for that same input. So this quantity is the **residual** or **error** for training example n : it is positive if the model under-predicts, negative if the model over-predicts, and exactly zero if the model predicts perfectly for that example.
- $\{\cdot\}^2$: we square the residual. Squaring serves two purposes. First, it makes every term non-negative, so that positive and negative errors do not cancel each other out when we sum over n (an error of $+3$ and an error of -3 are equally bad, and after squaring both become 9). Second, squaring makes the function smooth (infinitely differentiable) and, as we will see, makes the resulting optimisation problem have a single, clean, closed-form solution.
- $\sum_{n=1}^N$: this is a summation, meaning we add up the squared residual for every one of the N training examples, from $n = 1$ up to $n = N$. This total is often called the **sum of squared errors** (SSE).
- $\frac{1}{2}$: this constant factor out front does not change *where* the minimum of $E(w)$ occurs (multiplying a function by a positive constant does not move its minimiser), but it is included purely for algebraic convenience: as we will see in the next section, differentiating a square produces a factor of 2 from the chain rule, and this factor of $\frac{1}{2}$ exactly cancels it, leaving cleaner-looking equations.
- $E(w)$: the notation makes explicit that the error is a function of the weight vector w . The inputs x_n and targets t_n are fixed, given numbers coming from our training data; the only thing that varies, and that we are trying to choose optimally, is w .

In words: $E(w)$ measures the total squared mismatch, over the whole training set, between what the model predicts and what the true targets are, for a given setting of the weights w . Our task is to find the particular vector of weights, call it w^* , that makes $E(w)$ as small as possible. This is called **least squares** estimation, and w^* is called the **least squares solution**.

3 Calculus Background: Differentiating with Respect to a Vector

Before we can find the w that minimises $E(w)$, we need to know how to take a derivative of a scalar function with respect to a whole vector of variables at once, rather than one variable at a time.

Recall: Derivative of a Function of One Variable

If $f(x)$ is a function of a single real variable x , its derivative $f'(x) = \frac{df}{dx}$ tells us the instantaneous rate of change of f as x changes. A key fact we use repeatedly: at a point x^* where a smooth function f attains a local minimum (or maximum), the derivative must be zero, $f'(x^*) = 0$. This is because at a minimum, the function is momentarily flat: it is not increasing and not decreasing.

Recall: The Chain Rule

If $f(x) = g(h(x))^2$, i.e. f is the square of some inner function $h(x)$, then

$$\frac{df}{dx} = 2h(x) \cdot \frac{dh}{dx}.$$

In words: to differentiate “the square of something,” you bring down a factor of 2, keep the inside as it was, and then multiply by the derivative of the inside. This is exactly the rule we will use, since our

error function is built entirely out of squared quantities.

Common Point of Confusion

There are two common conventions for whether the gradient of a scalar with respect to a column vector is itself written as a column vector or as a row vector. In these notes (matching the layout used in the slides, where the derivative naturally appears multiplying $\phi(x_n)^T$, a row vector), we use the convention where the gradient of a scalar with respect to a column vector w is a *row* vector of the same width as w^T . This is purely a bookkeeping convention: it does not change any of the underlying mathematics, only how the results are laid out on the page.

For a fixed vector ϕ (not depending on w), differentiating the dot product $w^T\phi = w_0\phi_0 + w_1\phi_1 + \dots + w_{M-1}\phi_{M-1}$ with respect to each weight w_k leaves only ϕ_k behind (every other term is treated as a constant). Collecting these M partial derivatives into a row vector gives

$$\frac{d}{dw}(w^T\phi) = \phi^T.$$

4 Differentiating the Error Function

We now differentiate $E(w) = \frac{1}{2} \sum_{n=1}^N \{t_n - w^T\phi(x_n)\}^2$ with respect to w , term by term.

Fix one training example n and consider $\frac{1}{2}\{t_n - w^T\phi(x_n)\}^2$. This has exactly the “square of an inner function” shape from the chain rule box above, with the inner function being $h(w) = t_n - w^T\phi(x_n)$. By the chain rule,

$$\frac{d}{dw} \left[\frac{1}{2} \{t_n - w^T\phi(x_n)\}^2 \right] = \frac{1}{2} \cdot 2 \{t_n - w^T\phi(x_n)\} \cdot (-\phi(x_n)^T) = -\{t_n - w^T\phi(x_n)\} \phi(x_n)^T.$$

Notice how the $\frac{1}{2}$ from the error function has now cancelled exactly with the 2 that the chain rule produced, leaving a clean expression: this was the whole reason for including the $\frac{1}{2}$ in the definition of $E(w)$.

Summing this over all N training examples then gives

$$\frac{d}{dw} E(w) = \sum_{n=1}^N (-\{t_n - w^T\phi(x_n)\} \phi(x_n)^T) = - \sum_{n=1}^N \{t_n - w^T\phi(x_n)\} \phi(x_n)^T.$$

Common Point of Confusion

This derivation gives an overall minus sign, while the slides write

$$\frac{d}{dw} E(w) = \sum_{n=1}^N \{t_n - w^T\phi(x_n)\} \phi(x_n)^T.$$

Since we find the minimum by setting the derivative to zero, the overall sign is irrelevant: $A = 0$ and $-A = 0$ have the same solutions. Thus, either sign convention gives the same w^* . We follow the slides' convention for consistency and to avoid carrying an unnecessary minus sign.

So, adopting this convention, we take as our working formula for the gradient:

$$\frac{d}{dw} E(w) = \sum_{n=1}^N \{t_n - w^T\phi(x_n)\} \phi(x_n)^T$$

5 Setting the Gradient to Zero: the Normal Equations

Setting our gradient formula to the zero vector at the minimiser $w = w^*$ gives

$$\sum_{n=1}^N \{t_n - w^{*T}\phi(x_n)\} \phi(x_n)^T = 0. \quad (*)$$

Here, the “0” on the right-hand side is not the number zero, but the **zero row vector** $(0, 0, \dots, 0)$ of length M , since the left-hand side is itself a row vector (a sum of row vectors $\phi(x_n)^T$, each scaled by the scalar residual $\{t_n - w^{*T}\phi(x_n)\}$).

We now expand the term inside the summation using the distributive law, splitting the single sum into two separate sums:

$$\sum_{n=1}^N \{t_n - w^{*T}\phi(x_n)\} \phi(x_n)^T = \sum_{n=1}^N t_n \phi(x_n)^T - \sum_{n=1}^N w^{*T} \phi(x_n) \phi(x_n)^T.$$

In the second sum, $w^{*T}\phi(x_n)$ is just a scalar (a single number, the model’s prediction), and multiplying it into $\phi(x_n)^T$ just scales that row vector; since w^{*T} is the same fixed vector for every term in the sum (it does not depend on n), we can factor it out to the front of the summation:

$$\sum_{n=1}^N w^{*T} \phi(x_n) \phi(x_n)^T = w^{*T} \sum_{n=1}^N \phi(x_n) \phi(x_n)^T.$$

Common Point of Confusion

Be careful about the order of factors here. $\phi(x_n) \phi(x_n)^T$ (column vector times row vector, in that order) is very different from $\phi(x_n)^T \phi(x_n)$ (row vector times column vector). The dot product $\phi(x_n)^T \phi(x_n)$ is a 1×1 matrix, i.e. a single scalar number. But $\phi(x_n) \phi(x_n)^T$ is an $(M \times 1)$ matrix times a $(1 \times M)$ matrix, which gives an $M \times M$ **matrix**, not a scalar. This particular construction, a column vector multiplied by its own transpose, is called an **outer product**, and it will be central to what follows.

Substituting back, equation (*) becomes exactly what the slides label as Equation (1):

$$\sum_{n=1}^N t_n \phi(x_n)^T - w^{*T} \left(\sum_{n=1}^N \phi(x_n) \phi(x_n)^T \right) = 0. \quad (1)$$

Rearranging by moving the second term to the other side of the equation (adding it to both sides),

$$w^{*T} \left(\sum_{n=1}^N \phi(x_n) \phi(x_n)^T \right) = \sum_{n=1}^N t_n \phi(x_n)^T.$$

Recall: Solving a Matrix Equation by “Dividing”

If A is a square, invertible matrix and $bA = c$ (with b, c row vectors and A a matrix multiplying on the right of b), then we can solve for b by multiplying both sides on the right by A^{-1} , the **inverse** of A : $bAA^{-1} = cA^{-1}$, and since $AA^{-1} = I$ (the identity matrix, which leaves anything it multiplies unchanged), we get $b = cA^{-1}$. This is the matrix analogue of dividing both sides of an ordinary numerical equation by a nonzero number; matrices do not have a division operation, so multiplying by the inverse plays that role instead, and the order (left or right) matters because, unlike ordinary numbers, matrix multiplication is generally not commutative ($AB \neq BA$ in general).

Applying this idea, with $A = \sum_{n=1}^N \phi(x_n) \phi(x_n)^T$, we solve for w^{*T} by multiplying both sides on the right by A^{-1} :

$$w^{*T} = \left(\sum_{n=1}^N t_n \phi(x_n)^T \right) \left(\sum_{n=1}^N \phi(x_n) \phi(x_n)^T \right)^{-1}.$$

This is exactly the expression given in the slides. It is still written purely in terms of summations over the individual training examples; the next section converts it into pure matrix-vector notation, with no summation signs at all, by introducing the design matrix.

6 Linear Algebra Background: Matrices as Stacked Vectors

Recall: What is a Matrix?

A matrix is simply a rectangular grid of numbers, arranged into rows and columns. We say a matrix is $N \times M$ if it has N rows and M columns. A vector is just a special case of a matrix with only one column (a “column vector,” $N \times 1$) or only one row (a “row vector,” $1 \times M$).

Recall: Matrix Multiplication

If A is an $N \times K$ matrix and B is a $K \times M$ matrix, then the product $C = AB$ is an $N \times M$ matrix, and its entry in row i , column j is obtained by taking the dot product of row i of A with column j of B :

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}.$$

Two matrices can only be multiplied if the number of columns of the first equals the number of rows of the second; this shared number, K here, is sometimes called the “inner dimension,” and it disappears from the shape of the answer, leaving an $N \times M$ result.

We are now ready to define the **design matrix** Φ (the capital Greek letter Phi), which simply stacks the row vectors $\phi(x_n)^T$, for every training example $n = 1, \dots, N$, on top of each other to form a single big matrix:

$$\Phi = \begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_{M-1}(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_{M-1}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_{M-1}(x_N) \end{pmatrix}.$$

- Φ has N rows and M columns, i.e. it is an $N \times M$ matrix, matching the number of training examples (N) and the number of basis functions (M).
- **Row** n of Φ , read from left to right, is $(\phi_0(x_n), \phi_1(x_n), \dots, \phi_{M-1}(x_n))$. This is precisely the row vector $\phi(x_n)^T$ that we defined earlier: the transpose of the column vector of basis function outputs for the n -th training input.
- **Column** j of Φ , read from top to bottom, is $(\phi_j(x_1), \phi_j(x_2), \dots, \phi_j(x_N))^T$: it is the output of the single basis function ϕ_j , evaluated across every one of the N training inputs.
- In compact notation using the row-vector interpretation, we can write

$$\Phi = \begin{pmatrix} \phi(x_1)^T \\ \phi(x_2)^T \\ \vdots \\ \phi(x_N)^T \end{pmatrix}.$$

The whole point of the design matrix is that it packages every basis-function evaluation, across the entire training set, into one single matrix object, which lets us replace explicit summations over n with ordinary matrix multiplications. Concretely, two short facts about Φ let us do exactly that. Since Φ^T has columns $\phi(x_1), \phi(x_2), \dots, \phi(x_N)$ and Φ has rows $\phi(x_1)^T, \phi(x_2)^T, \dots, \phi(x_N)^T$, multiplying Φ^T by Φ pairs up the n -th column of Φ^T with the n -th row of Φ , for every n , and adds the results together. This gives

$$\Phi^T \Phi = \sum_{n=1}^N \phi(x_n) \phi(x_n)^T,$$

which is exactly the sum of outer products from our normal equations. The same idea applied to the row vector $t^T = (t_1, \dots, t_N)$ and the matrix Φ gives

$$t^T \Phi = \sum_{n=1}^N t_n \phi(x_n)^T.$$

We use both of these in the next section to rewrite our summation-based solution in pure matrix notation.

7 Assembling the Solution: the Normal Equations in Matrix Form

Substituting the two identities $\sum_n \phi(x_n)\phi(x_n)^T = \Phi^T\Phi$ and $\sum_n t_n\phi(x_n)^T = t^T\Phi$ into the row-vector solution we derived in Section 5,

$$w^{*T} = \left(\sum_{n=1}^N t_n \phi(x_n)^T \right) \left(\sum_{n=1}^N \phi(x_n)\phi(x_n)^T \right)^{-1} = t^T \Phi (\Phi^T \Phi)^{-1}.$$

To get the more commonly quoted **column-vector** form of the solution, we transpose both sides of this equation.

Recall: Transpose of a Product

For any matrices A and B with compatible shapes, $(AB)^T = B^T A^T$. Note carefully that the order of the two factors is reversed by the transpose. This rule extends to products of more than two matrices in the same way, e.g. $(ABC)^T = C^T B^T A^T$.

Applying this to $w^{*T} = t^T \Phi (\Phi^T \Phi)^{-1}$, and using the further fact that the transpose of an inverse is the inverse of the transpose, $(A^{-1})^T = (A^T)^{-1}$:

$$w^* = (w^{*T})^T = (t^T \Phi (\Phi^T \Phi)^{-1})^T = ((\Phi^T \Phi)^{-1})^T \Phi^T (t^T)^T = ((\Phi^T \Phi)^T)^{-1} \Phi^T t.$$

Finally, $\Phi^T \Phi$ is always a **symmetric** matrix, meaning $(\Phi^T \Phi)^T = \Phi^T \Phi$ (check: $(\Phi^T \Phi)^T = \Phi^T (\Phi^T)^T = \Phi^T \Phi$, using $(AB)^T = B^T A^T$ and $(\Phi^T)^T = \Phi$). A symmetric matrix equals its own transpose, and the inverse of a symmetric matrix is itself symmetric, so $((\Phi^T \Phi)^T)^{-1}$ is simply $(\Phi^T \Phi)^{-1}$, with no further transposition needed.

Putting it all together, we arrive at the celebrated **normal equations** solution for linear regression:

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T t$$

- w^* is the optimal $M \times 1$ weight vector, i.e. the setting of the weights that minimises the sum-of-squares error $E(w)$ over the entire training set.
- $\Phi^T \Phi$ is an $M \times M$ symmetric matrix built entirely from the inputs $x_1, \dots, x_N$ (through the basis functions); it does not depend on the targets t_n at all.
- $\Phi^T t$ is an $M \times 1$ column vector that combines both the inputs (through Φ) and the targets (through t).
- $(\Phi^T \Phi)^{-1}$ requires $\Phi^T \Phi$ to actually be invertible. This holds whenever Φ has M linearly independent columns (equivalently, whenever Φ has full column rank), which in practice usually requires $N \geq M$, i.e. at least as many training examples as parameters, together with the basis functions not being redundant with each other on the given data.

8 Summary

1. We define the model prediction as $y(x, w) = w^T \phi(x)$, a dot product between a learnable weight vector w and a fixed vector of basis-function outputs $\phi(x)$.
2. We measure the total mismatch between predictions and targets across the training set using the sum-of-squares error, $E(w) = \frac{1}{2} \sum_{n=1}^N \{t_n - w^T \phi(x_n)\}^2$.
3. Using the chain rule for differentiating a squared quantity, together with the linearity of differentiation, we compute the gradient of $E(w)$ with respect to w .
4. Setting this gradient to zero (a necessary condition for a minimum) gives the normal equations, first as a sum over training examples, $\sum_n t_n \phi(x_n)^T = w^{*T} \sum_n \phi(x_n)\phi(x_n)^T$.
5. We package all N basis-function vectors into a single $N \times M$ design matrix Φ , whose n -th row is $\phi(x_n)^T$.

6. Using the block-multiplication view of matrix products (a matrix product is a sum of outer products of matching columns and rows), we show $\sum_n \phi(x_n)\phi(x_n)^T = \Phi^T\Phi$ and $\sum_n t_n\phi(x_n)^T = t^T\Phi$.
7. Substituting these, transposing, and using the symmetry of $\Phi^T\Phi$, we arrive at the closed-form least squares solution $w^* = (\Phi^T\Phi)^{-1}\Phi^T t$.
8. We name the matrix $(\Phi^T\Phi)^{-1}\Phi^T$ the Moore-Penrose pseudo-inverse $\Phi^\dagger$ of Φ , which lets us write the solution compactly as $w^* = \Phi^\dagger t$, and which reduces to the ordinary matrix inverse Φ^{-1} whenever Φ happens to be square and invertible.